

Q.No :4

Question description

Problem Description:

For Engineering Second -year students, the instructor is organising a surprise test.

He's given you a new node is added before the given node of the given Linked List.

For example if the given Linked List is 5->10->15->20->25 and

we add an item 30 before node 15, then the Linked List becomes 5->10->30->15->20->25.

Since a Linked List is typically represented by the head of it, we have to traverse the list till node and then insert the node.

Constraints:

1 < arr <100

Input Format

First line contains the number of datas- N.

Second line contains N integers(the given Linked List).

Third line contains the node P before which the node to be inserted.

Fourth line contain the node X to be inserted.

Output Format

case 1 (node P found in Linked list) :

Display the final Linked List.

case 2 (node P not found in Linked list) :

Print Node not found!

Display the Linked list.

Logical Test Cases

Test Case 1

INPUT (STDIN)

4
115 210 116 110
116
17

EXPECTED OUTPUT

Linked List:->115->210->17->116->110

Test Case 2

INPUT (STDIN)

4
15 20 16 10
25
17

EXPECTED OUTPUT

Node not found!
Linked List:->15->20->16->10

Mandatory Test Cases

Test Case 1

KEYWORD

struct node

Test Case 2

KEYWORD

node *next;

Test Case 3

KEYWORD

*start

Test Case 4

KEYWORD

p2->next=p1;

Test Case 5

KEYWORD

void display()

Code:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *start = NULL;

void create(int n)
{
    struct node *newnode, *temp;
```

```

int data;

for(int i = 0; i < n; i++)
{
    scanf("%d", &data);

    newnode = (struct node *)malloc(sizeof(struct node));

    newnode->data = data;
    newnode->next = NULL;

    if(start == NULL)
    {
        start = newnode;
    }
    else
    {
        temp = start;

        while(temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newnode;
    }
}

void insertBefore(int P, int X)
{
    struct node *p1, *p2, *newnode;

    newnode = (struct node *)malloc(sizeof(struct node));
    newnode->data = X;
    newnode->next = NULL;

    if(start != NULL && start->data == P)
    {
        newnode->next = start;
        start = newnode;
        return;
    }

    p1 = start;
    p2 = start;

    while(p2 != NULL && p2->data != P)
    {
        p1 = p2;
        p2 = p2->next;
    }

    if(p2 == NULL)

```

```
{  
free(newnode);  
printf("Node not found!\n");  
return;  
}
```

```
newnode->next = p2;  
p1->next = newnode;  
p2->next = p1;  
p1->next = newnode;  
}
```

```
void display()  
{  
struct node *temp = start;
```

```
printf("Linked List:");
```

```
while(temp != NULL)  
{  
printf("->%d", temp->data);  
temp = temp->next;  
}
```

```
printf("\n");  
}
```

```
int main()  
{  
int n, P, X;
```

```
printf("Input value:\n");
```

```
scanf("%d", &n);
```

```
create(n);
```

```
scanf("%d", &P);  
scanf("%d", &X);
```

```
insertBefore(P, X);
```

```
printf("Output:\n");
```

```
display();
```

```
return 0;  
}
```

Output:

Test_case 1:

```
Input value:
4
115 210 116 110
116
17
Linked List:->115->210->17->116->110
```

Test_case 2:

```
Input value:
4
15 20 16 10
25
17
Node not found!
Linked List:->15->20->16->10
```